

# Building Forms

From plain HTML inputs to fully type-validated forms. Learn the limits of native forms, then build controlled, typed forms with clear validation and error messages.

## WHAT WE COVER

Native forms

Controlled inputs

Typed form state

Validation

Zod schemas

Error messages

Forms are how users give your app data — sign-ups, logins, checkout, adding a product. We'll start with plain HTML forms and see exactly where they fall short: everything is a string, there's no type safety, and no built-in validation. Then we'll rebuild them in React with TypeScript so the shape of the data is guaranteed, add validation, and show friendly error messages when the user types something wrong.

# Building Forms

A form is just inputs plus rules. Native HTML gives you the inputs; React + TypeScript give you **typed data** and **validation**. Today you'll go from a plain form to a type-safe, schema-validated one that tells the user precisely what to fix.

## CLASS AGENDA · ~2 HOURS

|      |                                           |       |
|------|-------------------------------------------|-------|
| 0:00 | Native HTML forms and their limits        | \$1-2 |
| 0:25 | Controlled inputs & typing the whole form | \$3-4 |
| 0:50 | Handling submit; manual validation        | \$5-6 |
| 1:15 | Schema validation with Zod                | \$7   |
| 1:35 | Showing errors & the project              | \$8-9 |
| 1:55 | Challenges & wrap-up                      | -     |

## ON THIS HANDOUT

- |                                |                                         |
|--------------------------------|-----------------------------------------|
| 01. Native HTML forms          | 06. Manual validation with typed errors |
| 02. The limits of native forms | 07. Schema validation with Zod          |
| 03. Controlled inputs (typed)  | 08. Showing errors to the user          |
| 04. Typing the whole form      | 09. Project: an Add-Product form        |
| 05. Handling submit            |                                         |

## PART 1 · Forms the Native Way

### 01 Native HTML forms

A form starts with plain HTML: a `<form>` wrapping `<label>` + `<input>` pairs and a submit button. Every input should have a matching label (good for accessibility).

```

<form>
  <label htmlFor="email">Email</label>
  <input id="email" name="email" type="email" />

  <label htmlFor="age">Age</label>
  <input id="age" name="age" type="number" />

  <button type="submit">Sign up</button>
</form>

```

TSX

- ▶ `<label htmlFor="email">` links to the input's `id` — clicking the label focuses the field.
- ▶ The `type` attribute ( `email` , `number` , `password` ) gives basic browser behaviour.
- ▶ This renders and works — but it's doing very little for you.

## 02 The limits of native forms

Native forms look fine until you actually need the data. Their weaknesses:

- ▶ **Everything is a string.** Even `type="number"` gives you the text `"25"` , not the number `25` .
- ▶ **No type safety.** Nothing describes the shape of your form data, so typos and missing fields aren't caught.
- ▶ **No real validation.** Browser checks are minimal and inconsistent; you can't express rules like “age must be 18+” or “passwords must match.”
- ▶ **No schema.** There's no single source of truth for what valid data looks like.
- ▶ **Manual everything.** Reading values, checking them, showing errors — all by hand, error-prone.

React gives us controlled inputs to capture values cleanly; TypeScript gives us the shape; and a validation schema gives us the rules. Let's build up to that.

## PART 2 · Controlled, Typed Forms

## 03 Controlled inputs (typed)

A **controlled input** keeps its value in React state: the state is the single source of truth, and every keystroke updates it. This is what lets us read, validate and reset form data reliably.

```
function EmailField() {
  const [email, setEmail] = useState<string>("");

  return (
    <input
      value={email} // value comes FROM state
      onChange={e => setEmail(e.target.value)} // typing updates state
    />
  );
}
```

- ▶ `value={email}` ties the input to state; `onChange` writes changes back.
- ▶ Now the current value always lives in state, ready to validate or submit.

## 04 Typing the whole form

Rather than one state per field, describe the whole form with an **interface** (Day 9) and hold it in a single typed state object.

```
interface SignupForm {
  name: string;
  email: string;
  age: number;
}

function Signup() {
  const [form, setForm] = useState<SignupForm>({ name: "", email: "", age: 0 });

  return (
    <form>
      <input value={form.name}
        onChange={e => setForm({ ...form, name: e.target.value })} />

      <input type="number" value={form.age}
        onChange={e => setForm({ ...form, age: Number(e.target.value) })} />
      { /* Number(...) converts the input's text into an actual number */ }
    </form>
  );
}
```

- ▶ Update one field immutably: copy the object with `...form`, then override that field.
- ▶ Convert numeric inputs with `Number(e.target.value)` so `age` is really a number.
- ▶ TypeScript now guarantees the form always has exactly these fields with the right types.

## 05 Handling submit

On submit, stop the browser's default page reload, then use the typed data.

```
function handleSubmit(e: React.FormEvent) {
  e.preventDefault(); // stop the native full-page reload
  console.log(form); // form is fully typed here
  // ...send it to an API, or dispatch to Redux (Day 14)...
}

// on the form element:
<form onSubmit={handleSubmit}> ... </form>
```

`e.preventDefault()` is essential — without it the page reloads and you lose everything, just like the form in Day 12's routing lesson.

## PART 3 • Validation

### 06 Manual validation with typed errors

Validation means checking the data against rules and collecting **errors**. A clean approach: a function that returns a typed errors object keyed by field name.

```
type Errors = Partial<Record<keyof SignupForm, string>>; // one message per field

function validate(data: SignupForm): Errors {
  const errors: Errors = {};
  if (!data.name.trim()) errors.name = "Name is required";
  if (!data.email.includes("@")) errors.email = "Enter a valid email";
  if (data.age < 18) errors.age = "You must be 18 or older";
  return errors;
}

function handleSubmit(e: React.FormEvent) {
  e.preventDefault();
  const found = validate(form);
  setErrors(found);
  if (Object.keys(found).length === 0) {
    console.log("All valid!", form);
  }
}
```

- ▶ `Partial<Record<keyof SignupForm, string>>` means “an optional message for each field” — fully typed.
- ▶ An empty errors object means the form is valid.

This works well, but for anything beyond a few fields you'll want a proper **schema** — one declaration that both validates and provides types.

## 07 Schema validation with Zod

Zod is a schema-validation library: you declare the rules once, and it both **checks the data at runtime** and **gives you the TypeScript type** for free. One source of truth for shape and rules.

```
npm install zod
```

TERMINAL

```
import { z } from "zod";

const signupSchema = z.object({
  name: z.string().min(1, "Name is required"),
  email: z.email("Enter a valid email"),
  age: z.number().min(18, "You must be 18 or older"),
});

// the TYPE is inferred FROM the schema -- no separate interface needed
type SignupForm = z.infer<typeof signupSchema>;
// -> { name: string; email: string; age: number }
```

TS

- ▶ Each field gets rules and a custom message: `.min(1, ...)`, `z.email(...)`, `.min(18, ...)`.
- ▶ `z.infer<typeof schema>` generates the type — change the schema, the type updates automatically.
- ▶ (Note: with older Zod v3 you'd write `z.string().email()`; v4 uses `z.email()`.)

To validate, call `safeParse` — it never throws, it returns success or a structured error you can turn into per-field messages:

```
const result = signupSchema.safeParse(form);

if (!result.success) {
  const fieldErrors = z.flattenError(result.error).fieldErrors;
  // { email: ["Enter a valid email"], age: ["You must be 18 or older"] }
  setErrors(fieldErrors);
} else {
  console.log("Valid data:", result.data); // result.data is typed as SignupForm
}
```

TS

## 08 Showing errors to the user

The final piece: display each error next to its field, and stop invalid data from being submitted. Good forms tell users exactly what to fix.

```

<form onSubmit={handleSubmit}>
  <label htmlFor="email">Email</label>
  <input id="email" value={form.email}
    onChange={(e) => setForm({ ...form, email: e.target.value })}
    className={errors.email ? "input input--error" : "input"} />
  {errors.email && <p className="error-text">{errors.email}</p>}

  <button type="submit">Sign up</button>
</form>

```

TSX

- ▶ Render the message only when it exists: `errors.email && <p>...`.
- ▶ Add an error class to highlight the invalid field (red border, etc.).
- ▶ Validate on submit for simplicity; later you can also validate on blur or as the user types.

#### TRY IT YOURSELF

Add a `password` and `confirmPassword` to the schema and show an error when they don't match.

## 09 Project: an Add-Product form

Build a typed, validated form for adding a product — the exact form you'll drop into the e-commerce app on Day 15.

### The schema (shape + rules + type)

```

import { z } from "zod";

const productSchema = z.object({
  name: z.string().min(2, "Name must be at least 2 characters"),
  price: z.number().positive("Price must be greater than 0"),
  category: z.string().min(1, "Choose a category"),
});

type ProductForm = z.infer<typeof productSchema>;

```

TS

## The component

```
function AddProduct() {
  const [form, setForm] = useState<ProductForm>({ name: "", price: 0, category: "" });
  const [errors, setErrors] = useState<Record<string, string[] | undefined>>({});

  function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    const result = productSchema.safeParse(form);
    if (!result.success) {
      setErrors(z.flattenError(result.error).fieldErrors);
      return;
    }
    console.log("Add this product:", result.data); // valid + typed
    setErrors({});
  }

  return (
    <form onSubmit={handleSubmit}>
      <input placeholder="Name" value={form.name}
        onChange={(e) => setForm({ ...form, name: e.target.value })} />
      {errors.name && <p className="error-text">{errors.name[0]}</p>}

      <input type="number" placeholder="Price" value={form.price}
        onChange={(e) => setForm({ ...form, price: Number(e.target.value) })} />
      {errors.price && <p className="error-text">{errors.price[0]}</p>}

      <button type="submit">Add product</button>
    </form>
  );
}
```

- ▶ One schema drives both validation and the form's type.
- ▶ Errors show under each field; only valid data gets through.
- ▶ On Day 15 the valid product is dispatched to the Redux store instead of logged.

## Practice Challenges

Strengthen your forms with a few of these:

- ▶ Build a login form (email + password) with Zod validation and error messages.
- ▶ Disable the submit button while any field is invalid.
- ▶ Add a `<select>` for category and validate that one is chosen.
- ▶ Show a success message and clear the form after a valid submit.
- ▶ Validate a single field on `onBlur` so users see errors as they move between fields.

## Conclusion & what's next

---

You can now build forms properly: controlled inputs backed by **typed state**, handled submits, and validation — either hand-written with typed errors or, better, with a **Zod schema** that gives you validation and types from one declaration. And you show users clear, specific error messages.

**Next class (Day 14): state management with Redux** — a central store for data that many components share, like the shopping cart we'll build on Day 15.

### QUICK RECAP

Native forms are all strings with no type safety or schema · use **controlled inputs** + a **typed form object** · **preventDefault** on submit · validate with a typed function or a **Zod schema** ( `safeParse` + `flattenError` ) · show per-field **error messages** so users know what to fix.